

Lecture 28 Heaps

Chapter 12 of textbook

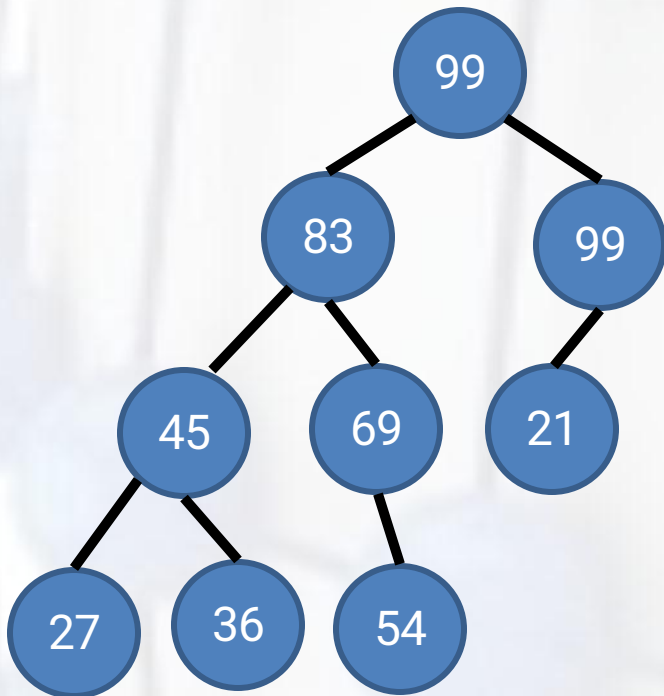
- Concepts of heaps
- Binary heaps
- Binomial heaps
- Fibonacci heaps
- Heap summary

1. Concept of Heaps

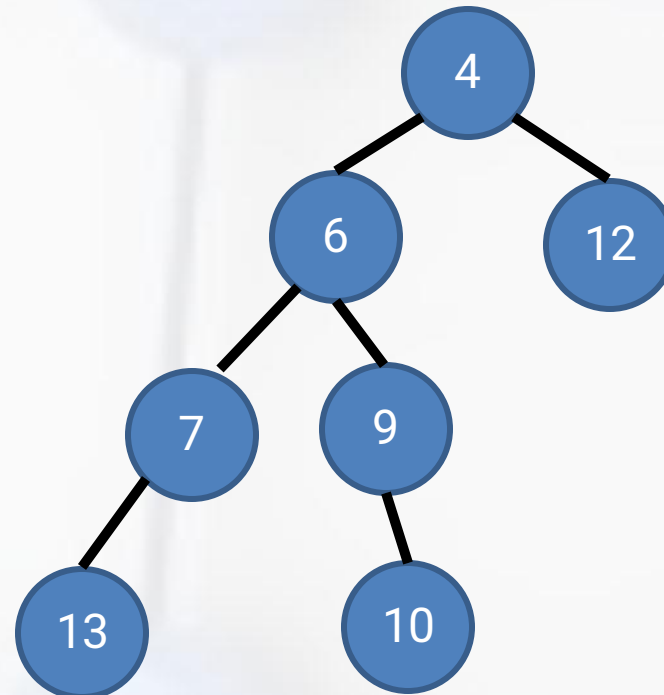
- A **heap** (or **heap-ordered tree**) is a tree-based data structure that satisfies the **heap property**:
 - each node has a key value
 - if A has child B, then $\text{key}(A) \geq \text{key}(B)$.
- This implies that the root node has the highest key value in the heap. Such a heap is called a **max-heap**.
- **min-heap**: if A has a child B, then $\text{key}(A) \leq \text{key}(B)$

Example of Heaps

max-heap



min-heap



Heap operations

find-max : find the node with maximum key value
from a max-heap, $O(1)$ time

find-min : find the node with minimum key value
from a min-heap, $O(1)$ time

Insert - insert a node into heap, maintain the property
of heap

delete - delete min/max key node from min-heap /
m-heap. maintain the property of heap
delete-min/extract-min, delete-max / extract-max

decrease-key – decrease key value of a node, maintain the heap
order

Applications of Heaps

Heap-sort: insert elements into heap, find-min/find-min delete get one after another, return a sorted list of elements.

Used to implement priority queue. Heap Implemented priority queues are used in Graph algorithms like Prim's Algorithm and Dijkstra's algorithm for shortest path

2. Binary Heaps

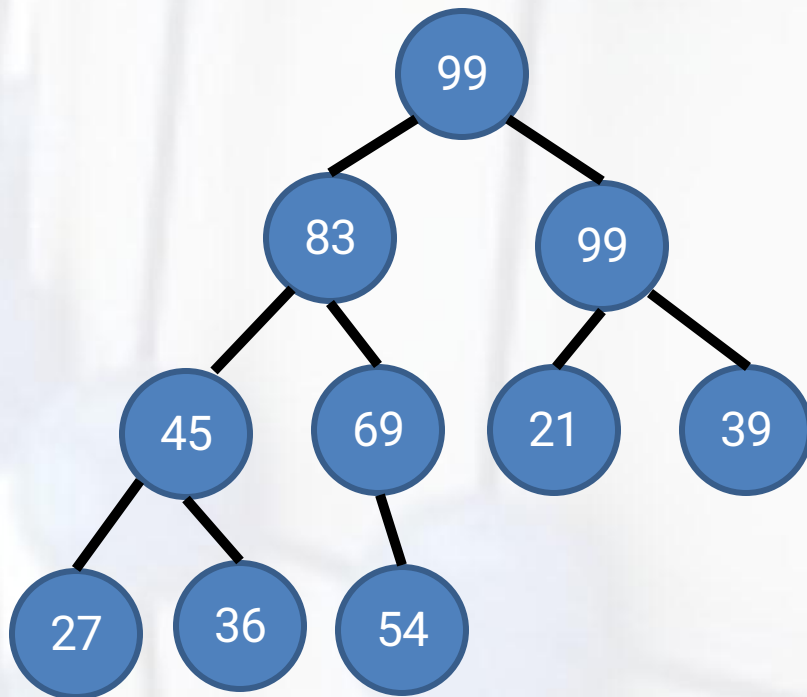
Binary heap is heap of **complete binary tree structure**

Complete binary tree

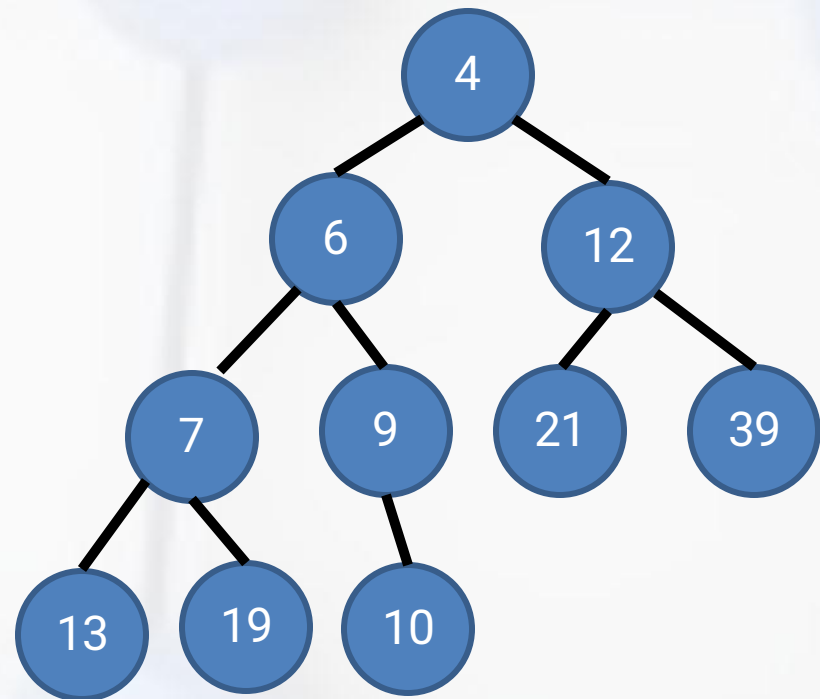
- all levels of the tree, except possibly the last one (deepest) are fully filled
- if the last level of the tree is not complete, the nodes of that level are filled from left to right.

Binary Heaps

Binary-max-heap

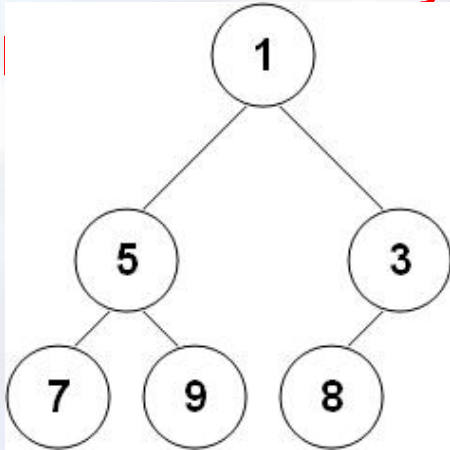


Binary min-heap



Array Representation of Binary Heaps

- A complete binary tree of n element can be represented by an array of n elements
 - Order the complete binary tree node data elements in breadth-first and left-first order, and put the elements into array in the same order. The root node has index 0.
 - The node of index i has children indices at $2i + 1$ and $2i + 2$, and parent at index $(i - 1) \div 2$
- The height of the complete binary tree is $\log_2 n$
- In case binary heap, the element at index 0 is the



Node	1	5	3	7	9	8
Index	0	1	2	3	4	5

Insertion in a Binary Heap

Algorithm: Insert element to binary max-heap H

Step 1. add the new element node to the bottom left H

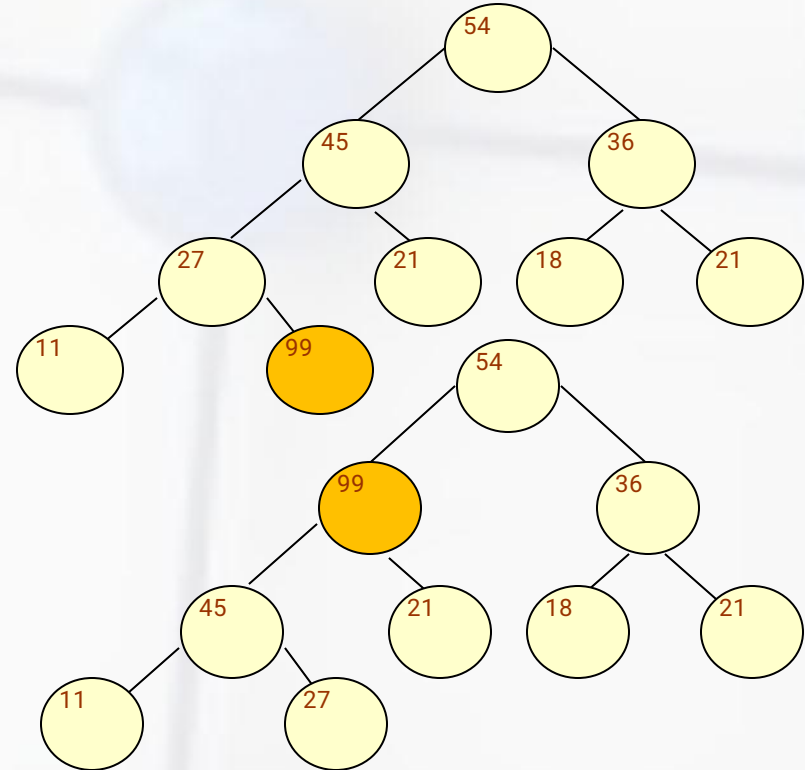
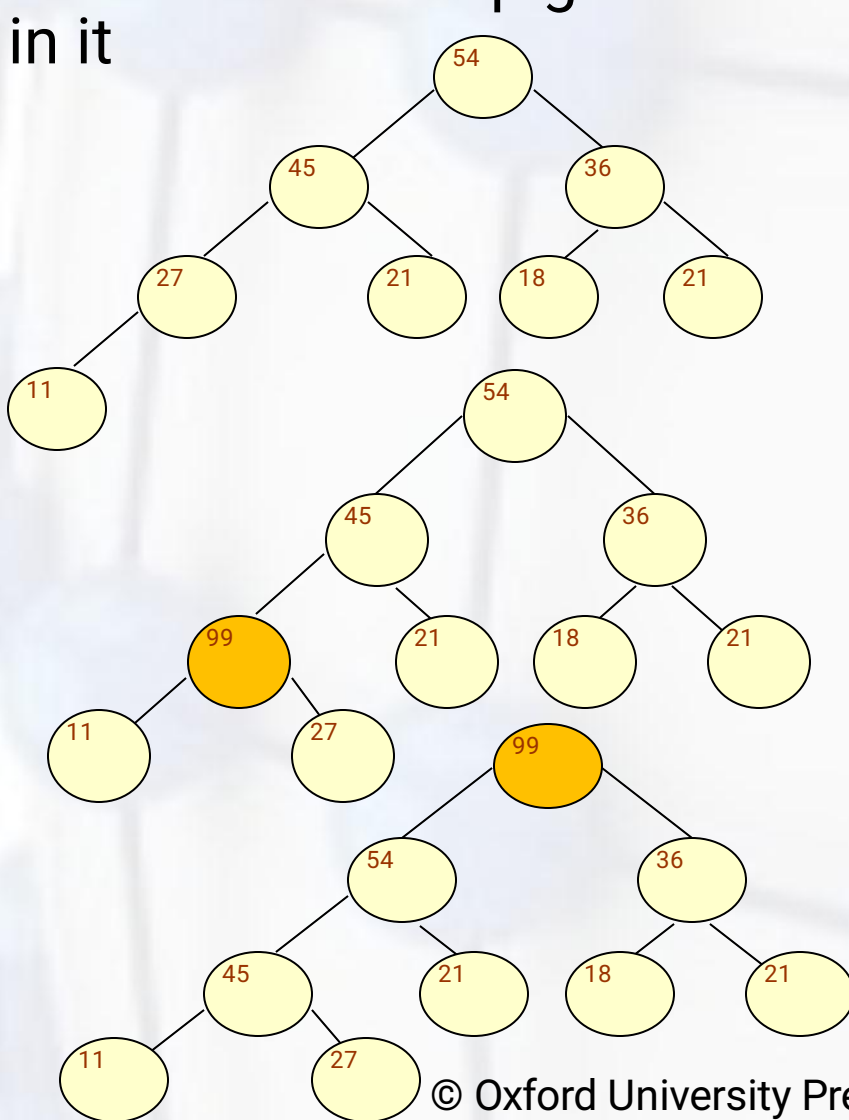
Step 2. If the new node value is bigger than its parent, swap their values. Set the parent node as new node, go to step 2.

Step 3. else exit.

Step 2 is called heapify

Example of Insertion in a Binary Heap

- Consider the heap given below and insert 99 in it



Insertion in a Binary Heap

Algorithm to insert a value in the heap

Step 1: [Add the new value and set its POS]

SET $N = N + 1$, $POS = N$

Step 2: SET $HEAP[N] = VAL$

Step 3: [Find appropriate location of VAL]

Repeat Steps 4 and 5 while $POS < 0$

Step 4: SET $PAR = POS/2$

Step 5: IF $HEAP[POS] \leq HEAP[PAR]$, then

Goto Step 6.

ELSE

SWAP $HEAP[POS]$, $HEAP[PAR]$

$POS = PAR$

[END OF IF]

[END OF LOOP]

Step 6: Return

Time complexity: ? Space complexity: ?

Deletion in a Binary Heap

- An element is always deleted from the root of the heap. So, deleting an element from the heap is done in two major steps.

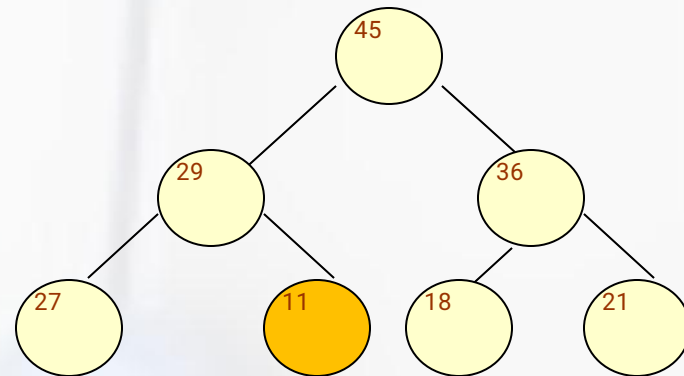
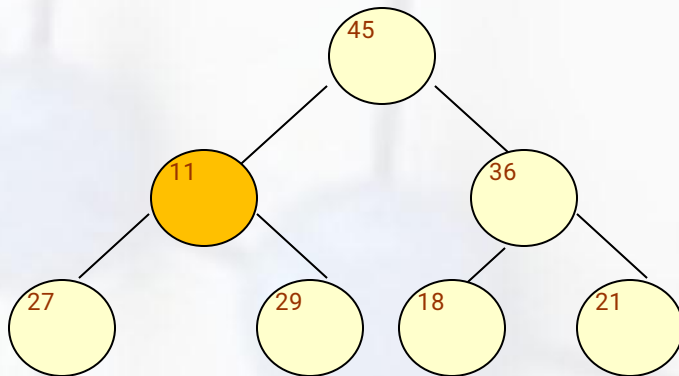
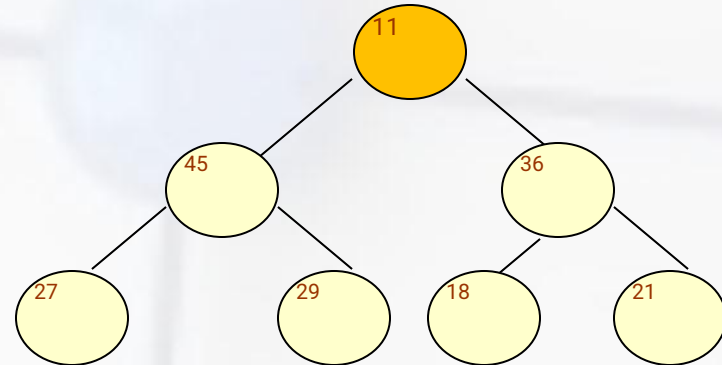
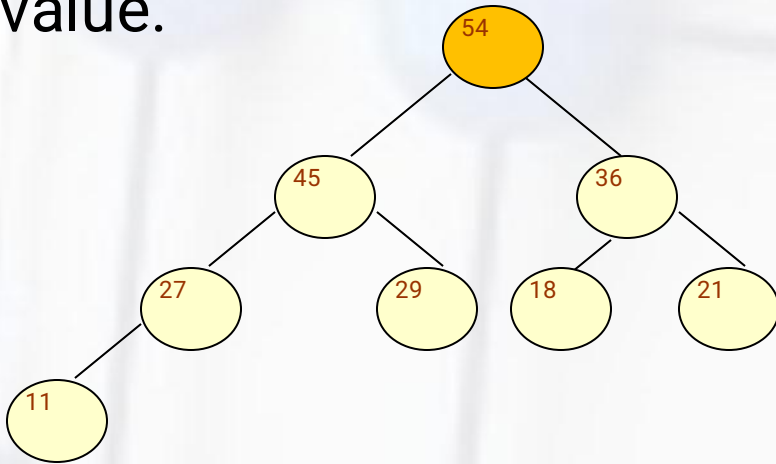
Step 1. Replace the root node's value with the last node's value so that H is still a complete binary tree but not necessarily a heap.

Step 2. Delete the last node.

Step 3. Sink down the new root node's value so that H satisfies the heap property. In this step, interchange the root node's value with its child node's value (whichever is largest among its children).

Deletion in a Binary Heap

- Consider the heap H given below and delete the root node's value.



Deletion in a Binary Heap

Algorithm to delete the root element from the heap

DELETE_HEAP(HEAP, N, VAL)

Step 1: [Remove the last node from the heap]

SET LAST = HEAP[N], SET N = N - 1

Step 2: [Initialization]

SET PTR = 0, LEFT = 1, RIGHT = 2

Step 3: SET HEAP[PTR] = LAST

Step 4: Repeat Steps 5 to 7 while LEFT <= N

Step 5: IF HEAP[PTR] >= HEAP[LEFT] AND HEAP[PTR] >= HEAP[RIGHT],
then Go to Step 8
[END OF IF]

Step 6: IF HEAP[RIGHT] <= HEAP[LEFT], then
SWAP HEAP[PTR], HEAP[LEFT]
SET PTR = LEFT

ELSE
SWAP HEAP[PTR], HEAP[RIGHT]
SET PTR = RIGHT

[END OF IF]

Step 7: SET LEFT = 2 * PTR and RIGHT = LEFT + 1

[END OF LOOP]

Step 8: Return

Time complexity: ? Space complexity: ?